# HANDBOOK

## Artificial Intelligence Assistant

Grade X – Vocational Education

**Python • NumPy • Pandas • Neural Networks • AI Project**

A textbook-style learning resource with examples, diagrams, code and review questions

Prepared with reference to the supplied Grade X study material

# Contents

**1 Python Programming** **1**
1.1 Control structures in Python . . . 1
1.1.1 Selection with `if`, `if...else` and `if...elif...else` . . . 1
1.1.2 Repetition using loops . . . 1
1.1.3 Loop control and common errors . . . 1
1.2 Functions in Python . . . 2
1.2.1 Parameters, return values and scope . . . 3
1.2.2 Designing a useful function . . . 3
1.2.3 Tracing a program step by step . . . 3
1.2.4 Writing readable and maintainable code . . . 3
1.2.5 Testing with normal, boundary and invalid values . . . 4

**2 Data Science with NumPy** **5**
2.1 Introduction to NumPy . . . 5
2.1.1 Array properties . . . 5
2.2 Array manipulation . . . 5
2.2.1 Indexing and slicing . . . 6
2.2.2 Broadcasting and practical cautions . . . 6
2.2.3 Joining, splitting and copying . . . 6
2.3 Array computation . . . 6
2.3.1 Why vectorised operations are useful . . . 6
2.3.2 Choosing an array structure . . . 7
2.3.3 Missing values and unusual measurements . . . 7
2.3.4 Efficiency and memory . . . 7
2.3.5 Practical exercise: summarising measurements . . . 7

**3 Data Analysis** **9**
3.1 Introduction to Pandas . . . 9
3.2 Working with Series and DataFrames . . . 9
3.2.1 Selecting, sorting and grouping records . . . 9
3.2.2 Cleaning and summarising data . . . 10
3.3 Data visualisation using Matplotlib . . . 10
3.3.1 Choosing a chart for the question . . . 11
3.3.2 Understanding records and data types . . . 11
3.3.3 Filtering records carefully . . . 11
3.3.4 Grouping and summarising . . . 12
3.3.5 Reproducible data cleaning . . . 12

**4 Neural Networks and Machine Learning Tools** **13**
4.1 Artificial Neural Networks . . . 13
4.1.1 Training and prediction . . . 13
4.1.2 Weights, activation functions and loss . . . 13
4.1.3 Overfitting and generalisation . . . 13

4.2 Applications of neural networks . . . . . 14
4.2.1 Responsible use . . . . . 14
4.2.2 Privacy, fairness and human oversight . . . . . 14
4.3 Machine learning tools . . . . . 14
4.3.1 How a neural network makes a prediction . . . . . 14
4.3.2 Training, validation and testing . . . . . 15
4.3.3 Classification and regression . . . . . 15
4.3.4 Limitations of neural networks . . . . . 15
4.3.5 Practical comparison exercise . . . . . 15

**5 AI Project** **16**
5.1 Project guidelines . . . . . 16
5.1.1 A practical project cycle . . . . . 16
5.2 Project formats and documentation . . . . . 16
5.2.1 Choosing evaluation measures . . . . . 16
5.3 Project review . . . . . 16
5.4 Sample project: student marks analysis . . . . . 17
5.4.1 Planning the work and presenting results . . . . . 17
5.4.2 Defining a measurable project objective . . . . . 17
5.4.3 Data permissions and documentation . . . . . 18
5.4.4 A practical project schedule . . . . . 18
5.4.5 Interpreting results honestly . . . . . 18
5.4.6 Project report structure . . . . . 18
5.4.7 Reflection after completion . . . . . 18

Chapter 1

# Python Programming

## 1.1 Control structures in Python

A Python program normally executes statements in the order in which they appear. Control structures let a program choose between alternatives or repeat a set of instructions. These structures are important when a solution must respond to different inputs rather than follow one fixed path.

### 1.1.1 Selection with `if`, `if...else` and `if...elif...else`

An `if` statement tests a condition. If the condition is true, its indented block runs. An `if...else` statement provides an alternative when the condition is false. An `if...elif...else` ladder checks several conditions in sequence; the first true branch is executed.

```
marks = int(input("Enter marks: "))
if marks >= 80:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Needs improvement")
```

Indentation is part of Python syntax. Statements belonging to a branch must be indented consistently. Conditions can use comparison operators such as ==, !=, >, <, >= and <=, along with logical operators `and`, `or` and `not`.

### 1.1.2 Repetition using loops

A `for` loop iterates over items in a sequence or over values produced by `range()`. A `while` loop repeats while its condition remains true. Every loop should make progress toward termination; otherwise, a `while` loop can continue indefinitely.

```
total = 0
for number in range(1, 6):
    total += number
print("Total:", total)
```

The program adds the integers from 1 through 5. The `range(1, 6)` expression stops before 6, so the values are 1, 2, 3, 4 and 5. A nested loop is a loop placed inside another loop and is useful for tables, patterns and two-dimensional data.

### 1.1.3 Loop control and common errors

The `break` statement stops the nearest loop, while `continue` skips the remaining statements in the current iteration and moves to the next one. These statements should be used only when they make the program easier to follow. A common error is an off-by-one mistake, in which a loop repeats one time too many or too few. Carefully check the starting value, stopping condition and update step.

```
for number in range(1, 8):
    if number == 5:
```

```
        continue
    print(number)
```

This example prints 1, 2, 3, 4, 6 and 7. The value 5 is skipped, but the loop continues. Testing a loop with a small set of known values is a useful way to confirm that it behaves as intended.

When a program receives data from a user, it should also consider invalid input. For example, converting a word to an integer with `int()` raises an error. A more robust program can validate input or handle a conversion error instead of terminating unexpectedly.

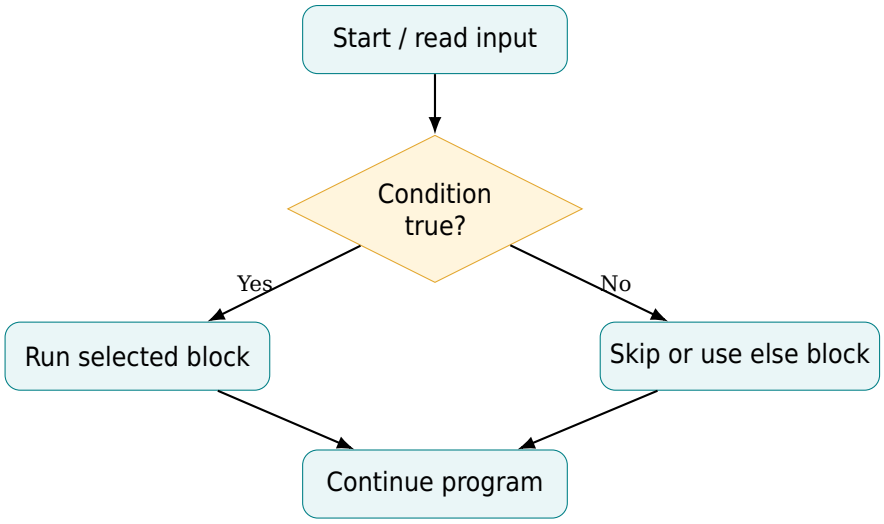



*Figure 1.1.* A simplified decision flow for a conditional statement.

## 1.2 Functions in Python

A function is a named, reusable block of code that performs a task. Functions help divide a larger problem into smaller parts, reduce repetition and make a program easier to test. Python includes built-in functions such as `print()`, `len()` and `sum()`; programmers can also define their own functions with `def`.

```
def mean(values):
    if len(values) == 0:
        return None
    return sum(values) / len(values)

marks = [72, 81, 67, 90]
print(mean(marks))
```

The function accepts a list as an argument and returns its arithmetic mean. The `return` statement sends a result back to the calling code. Parameters are names in a function definition; arguments are the values supplied when it is called. A local variable created inside a function is normally available only within that function.

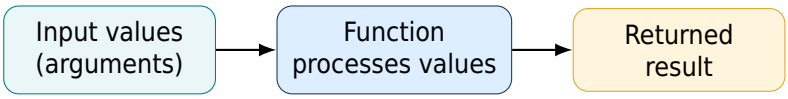



*Figure 1.2.* A function receives arguments, performs a task and may return a result.

### 1.2.1 Parameters, return values and scope

Functions may accept zero or more parameters. A function that returns a value can be used in an expression, while a function that only performs an action may return `None`. Variables defined inside a function have local scope. Clear names, short functions and appropriate validation improve readability and reliability.

### 1.2.2 Designing a useful function

A well-designed function usually has one clear responsibility. Its name should describe the task, its parameters should represent the information needed, and its return value should be predictable. Repeating the same calculation in several places makes later changes harder; placing that calculation in one function helps keep results consistent.

```
def percentage(score, maximum):
    if maximum <= 0:
        raise ValueError("Maximum must be positive")
    return (score / maximum) * 100

print(percentage(45, 50))
```

The function checks that the maximum possible score is positive before performing the calculation. This is an example of input validation. In larger programs, functions can be tested with normal values, boundary values and invalid values. A function should also be documented when its purpose or expected inputs are not obvious.

### 1.2.3 Tracing a program step by step

Tracing means following a program’s statements in the order in which they execute and recording how important variables change. It is useful when the output differs from what was expected. Start with the initial values, follow each condition, and update variables only when the relevant statement executes. A small trace table written in a notebook can help beginners reason about loops without relying on guesswork.

Consider a counter that begins at zero and increases once for each item processed. If the update statement is accidentally placed outside the loop, the counter may not represent the number of items. Similarly, changing the order of statements can change the result. Reading the code line by line and testing a small example makes such mistakes easier to identify.

### 1.2.4 Writing readable and maintainable code

Readable code uses meaningful names, consistent indentation and short blocks that have a clear purpose. A name such as `total_marks` communicates more than a single-letter name such as `x`. Comments should explain the reason for a decision or a non-obvious step rather than repeat every line in English. Blank lines can separate logical parts of a program, but excessive spacing makes a short program harder to follow.

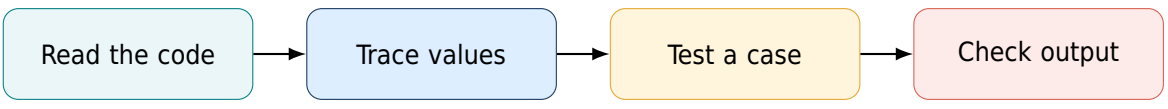



*Figure 1.3.* A simple method for tracing and checking a program.

### 1.2.5 Testing with normal, boundary and invalid values

A normal test checks an expected everyday input. A boundary test checks a value near a limit, such as marks equal to the minimum passing score. An invalid-input test checks what happens when the program receives a negative number where only positive values are permitted, or text where a number is expected. Testing all three kinds of input can reveal problems that one successful example would not show.

For example, a percentage function should be tested with a score of zero, a score equal to the maximum, and a maximum value of zero. The last case should be rejected because division by zero is undefined. Testing does not prove that a program has no defects, but it provides evidence that the program behaves as expected for the cases considered.

## Review Questions

1. Distinguish between `if...else` and `if...elif...else`.
2. What is the difference between a `for` loop and a `while` loop?
3. Write a function that returns the larger of two numbers.
4. Explain why indentation matters in Python.

Chapter 2

# Data Science with NumPy

## 2.1 Introduction to NumPy

NumPy is a Python library for numerical computing. Its central data structure is the multidimensional array, commonly called an `ndarray`. Compared with ordinary Python lists, NumPy arrays support convenient element-wise operations and efficient numerical processing. NumPy is widely used as a foundation for scientific computing and data analysis.

```
import numpy as np
marks = np.array([72, 81, 67, 90])
print(marks.mean())
print(marks.max())
print(marks.min())
```

The import statement makes NumPy available under the short name `np`, a common convention. The array stores four numerical values. Its methods calculate the mean, maximum and minimum without requiring a manually written loop.

### 2.1.1 Array properties

The `ndim` attribute reports the number of dimensions, `shape` reports the size along each dimension, `size` gives the total number of elements, and `dtype` indicates the element data type. A one-dimensional array can represent a sequence; a two-dimensional array can represent rows and columns.

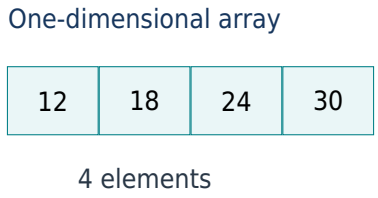



*Figure 2.2.* A one-dimensional NumPy array stores an ordered sequence of values.

```
table = np.array([[10, 20, 30],
                  [40, 50, 60]])
print(table.ndim)   # 2
print(table.shape)  # (2, 3)
print(table.size)   # 6
```

## 2.2 Array manipulation

Arrays can be reshaped, joined, split, indexed and sliced. A reshape operation changes the arrangement of elements without changing their total number. Indexing selects particular elements; slicing selects a range. The requested shape must be compatible with the number of values in the array.

```
values = np.array([1, 2, 3, 4, 5, 6])
matrix = values.reshape(2, 3)
print(matrix)
print(matrix[:, 1])
```

The expression `matrix[:, 1]` selects the second column. In a slice, a colon represents a range along an axis. Understanding array shape is essential when combining data from multiple sources.

### 2.2.1 Indexing and slicing

NumPy uses zero-based indexing, which means the first element has index 0. Negative indices count from the end of an array. Slicing selects a range using the form `start:stop:step`; the stop position is excluded. For a two-dimensional array, separate index expressions identify the row and column.

```
scores = np.array([55, 68, 72, 89, 94])
print(scores[0])        # first value
print(scores[-1])       # last value
print(scores[1:4])      # values at positions 1, 2, 3
print(scores[scores >= 70])
```

The last line uses a Boolean condition to select values that are at least 70. Such filtering is useful when exploring measurements, marks or other numerical records. The original data should be preserved when an analysis requires a record of the unmodified values.

### 2.2.2 Broadcasting and practical cautions

Broadcasting allows NumPy to combine arrays of compatible shapes. For example, adding a single number to an array adds that number to every element. More complex broadcasting follows shape rules, so dimensions should be checked when an operation produces an unexpected result. Arrays should also use appropriate numeric types: converting a decimal measurement to an integer can discard its fractional part.

### 2.2.3 Joining, splitting and copying

Functions such as `np.concatenate()` join arrays along a compatible axis. Splitting functions divide arrays into smaller arrays. A view can share underlying data with another array, whereas `copy()` creates independent data. Changing a shared view may therefore affect the original array.

## 2.3 Array computation

NumPy supports arithmetic, comparison, logical, statistical and mathematical operations. Many operations are performed element by element. Broadcasting allows arrays with compatible shapes to participate in an operation without manually copying values.

```
a = np.array([1, 2, 3])
print(a * 2)        # [2 4 6]
print(a + 10)       # [11 12 13]
print(np.sqrt(a))   # square roots
```

Common statistical functions include `sum()`, `mean()`, `median()`, `min()` and `max()`. Mathematical utilities include trigonometric functions and conversions between degrees and radians. The result type depends on the operation and input types.

### 2.3.1 Why vectorised operations are useful

A vectorised operation expresses a calculation for an entire array instead of describing each element with a Python loop. This often makes numerical code shorter and can improve performance. The expression should still be chosen for clarity: a com-

pact calculation is not helpful if its meaning is difficult to understand.

For example, if a set of temperatures is stored in an array in degrees Celsius, the conversion to Fahrenheit can be applied to every value using `temperatures * 9 / 5 + 32`. The formula is applied element by element. Before interpreting a result, check the units, the range of values and whether any missing or unusual measurements need attention.

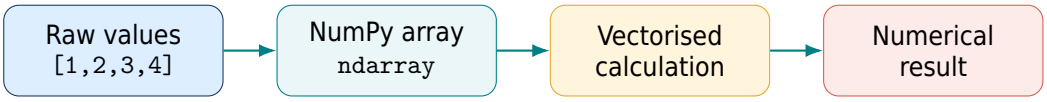



*Figure 2.1.* A typical numerical data-processing path.

### 2.3.2 Choosing an array structure

A one-dimensional array is suitable for a single sequence of measurements, such as daily temperatures. A two-dimensional array can represent a grid or a table of numerical values. Higher-dimensional arrays can represent batches of images or measurements collected across several dimensions. The structure should reflect the meaning of the data, not merely the shape that is convenient to type.

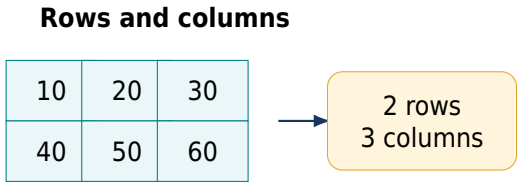



*Figure 2.3.* A two-dimensional array can represent numerical data arranged in rows and columns.

### 2.3.3 Missing values and unusual measurements

A numerical array can contain values that are mathematically valid but incorrect for the real problem. For instance, a temperature of 900 degrees may be a valid number in a computer but an implausible reading for a classroom thermometer. A negative mark may be outside the permitted range for an examination. Range checks and knowledge of the measurement context help identify such cases.

Missing values require a deliberate decision. If a sensor did not record a reading, replacing it with zero may falsely imply that the measured quantity was zero. Depending on the purpose, the missing value may be excluded from a calculation, investigated, or replaced using a justified method. Always record the choice and consider how it could affect the conclusion.

### 2.3.4 Efficiency and memory

NumPy is designed for numerical work with arrays and often performs large element-wise calculations more efficiently than manually looping through ordinary Python lists. The actual performance depends on the size of the data, the operation and the computer. For a tiny list, readability matters more than trying to optimise every operation. For large datasets, choosing an appropriate data type and avoiding unnecessary copies can reduce memory use.

### 2.3.5 Practical exercise: summarising measurements

Create an array containing five fictional daily temperatures. Print the minimum, maximum and mean. Then select values above the mean and explain what the filtered result represents. Change one value to an implausible number and discuss

how a range check could identify it. The exercise should include the code, its output and a short interpretation in words.

## Review Questions

1. What do `shape`, `ndim` and `dtype` describe?
2. Explain indexing and slicing.
3. What is the difference between a view and a copy?
4. State two advantages of NumPy arrays.

Chapter 3

# Data Analysis

## 3.1 Introduction to Pandas

Pandas is an open-source Python library used to organise, clean, analyse and summarise data. It is built around two important data structures: `Series`, a labelled one-dimensional structure, and `DataFrame`, a two-dimensional structure with rows and columns. Pandas is useful when data is arranged as a table, such as attendance records, marks, product lists or survey responses.

```
import pandas as pd
students = pd.DataFrame({
    "Name": ["Asha", "Ravi", "Meena"],
    "Marks": [78, 85, 91],
    "Class": ["X", "X", "X"]
})
print(students)
print(students["Marks"].mean())
```

A DataFrame can contain columns of different data types. Column labels help identify fields, while the row index identifies records. Data should be checked for missing values, duplicate records and inconsistent formats before analysis.

| Name | Marks | Class |
|---|---|---|
| Asha | 78 | X |
| Ravi | 85 | X |

*Figure 3.2.* A simplified DataFrame with named columns and individual records.

## 3.2 Working with Series and DataFrames

A Series stores values with associated labels. A DataFrame groups multiple columns into a table. Pandas can create a Series from a list or dictionary, select columns, filter rows, sort values and calculate summaries.

```
marks = pd.Series([72, 88, 64, 95])
print(marks.mean())
print(marks[marks >= 80])
```

Boolean filtering produces the values that satisfy a condition. In a DataFrame, a similar condition can be used to select records. Labels and positions should not be confused: label-based selection and integer-position selection have different purposes.

### 3.2.1 Selecting, sorting and grouping records

Pandas makes it possible to select a subset of rows using a condition, sort records by a column and calculate summaries for groups. These operations help turn a large table into information that answers a particular question. A filter should be linked to a clear question, such as identifying records above a chosen threshold.

```
high_scores = students[students["Marks"] >= 80]
sorted_students = students.sort_values("Marks", ascending=False)
print(high_scores)
print(sorted_students)
```

The first expression selects students whose marks are at least 80; the second orders the records from highest marks to lowest. In real projects, the meaning of each column and the unit of measurement should be recorded so that the output is interpreted correctly.

### 3.2.2 Cleaning and summarising data

Data cleaning may include handling missing values, correcting data types, removing duplicates and standardising text. The correct treatment depends on what a missing or unusual value means in the real context. Deleting records without review can distort the analysis.

A missing value can mean that information was not collected, was not applicable or was accidentally omitted. These situations do not always have the same meaning. Depending on the task, a missing value may be left as missing, investigated with the data owner, or handled using a documented method. Replacing all missing values with zero is not always appropriate because zero may be a valid measurement.

Duplicate records also need careful review. Some duplicates are accidental copies, while others represent legitimate repeated events. Before removing them, check which fields define a unique record. Cleaning decisions should be documented so that another person can understand how the original data was transformed.

```
print(students.head())
print(students.info())
print(students.isna().sum())
print(students.describe())
```

The `head()` method previews records, `info()` summarises columns and data types, `isna().sum()` counts missing values by column, and `describe()` gives descriptive statistics for suitable columns.

## 3.3 Data visualisation using Matplotlib

Data visualisation presents information graphically so that patterns, comparisons and changes can be understood. Matplotlib is a Python library for creating charts. The chart type should match the question: bar charts compare categories, line charts show change over an ordered sequence, and scatter plots explore relationships between numerical variables.

```
import matplotlib.pyplot as plt
subjects = ["English", "Maths", "Science"]
marks = [76, 88, 81]
plt.bar(subjects, marks)
plt.title("Marks by Subject")
plt.xlabel("Subject")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

A clear chart needs a meaningful title, labelled axes and a scale that does not mislead. Colours and decorative effects should support interpretation rather than distract from the data. Charts should be checked against the source values before

conclusions are drawn.

### 3.3.1 Choosing a chart for the question

A bar chart is suitable for comparing a small number of categories. A line chart is often appropriate for a measurement recorded over time, such as monthly rainfall. A scatter plot can help reveal whether two numerical variables appear to be related. A histogram shows the distribution of a numerical variable by grouping values into intervals. No chart proves a cause-and-effect relationship by itself.

A chart should include a title that states what is being shown, labels that explain the axes, and units where relevant. The scale should not exaggerate small differences. When comparing groups, use consistent scales and explain any filtering or exclusions. A well-labelled chart helps readers understand both the result and its limits.

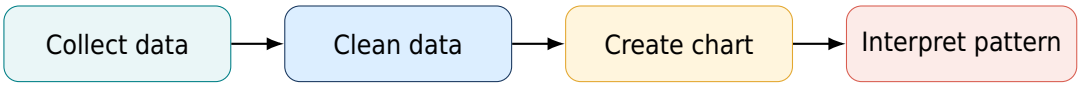



*Figure 3.3.* A simple sequence for producing a trustworthy data visualisation.

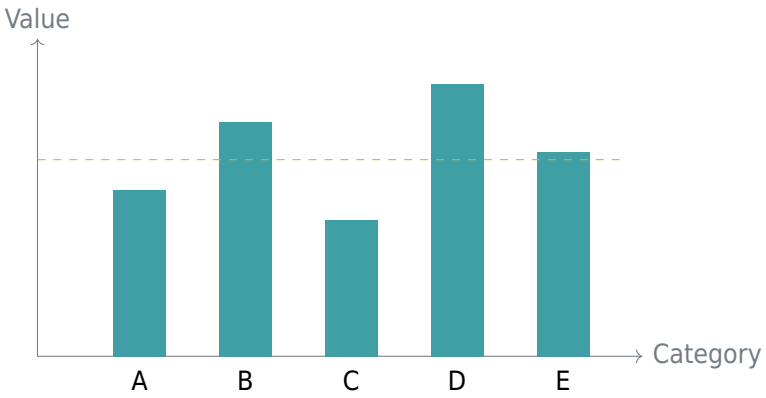



*Figure 3.1.* A schematic bar chart showing how categories can be compared.

### 3.3.2 Understanding records and data types

A data table represents observations as rows and properties as columns. In a student marks dataset, one row may represent one learner and columns may represent a name, subject, marks and examination date. A column should have a consistent meaning. If marks are stored partly as numbers and partly as text, calculations and comparisons may behave unexpectedly.

A Pandas Series is a one-dimensional labelled structure. A DataFrame combines rows and columns and can contain columns with different data types. The index labels rows, while column names identify fields. An index is not necessarily a unique identity for a real-world person or item; if a reliable identifier is required, it should be designed and checked separately.

### 3.3.3 Filtering records carefully

Filtering selects records that satisfy a condition. Multiple conditions can be combined, but each condition should be enclosed in parentheses when using Pandas Boolean expressions. The symbols `&` and `|` represent element-wise AND and OR; Python's `and` and `or` are not substitutes for these operators in a Series expression.

```python
import pandas as pd

students = pd.DataFrame({
    "Name": ["Asha", "Ravi", "Meena", "Imran"],
    "Marks": [78, 91, 84, 65],
    "Present": [True, True, False, True]
})
high = students[students["Marks"] >= 80]
print(high[["Name", "Marks"]])
```

The expression creates a smaller DataFrame containing only records with marks of at least 80, and then displays selected columns. In a real report, the filter condition should be stated clearly so readers understand which records were included.

### 3.3.4 Grouping and summarising

Grouping is useful when the question asks for a summary by category, such as average marks by subject or the number of records in each region. A group summary can hide differences between individual observations, so it should be interpreted at the correct level. When group sizes differ, a total and an average answer different questions; choose the statistic that fits the purpose.

### 3.3.5 Reproducible data cleaning

A reproducible cleaning process records what was changed and why. Keep an untouched copy of the original data where appropriate, and perform cleaning steps in a clear order. Check the number of rows before and after removing duplicates, confirm that conversions succeeded, and inspect a sample of the cleaned result. These checks reduce the risk of silently removing valid records or introducing errors.

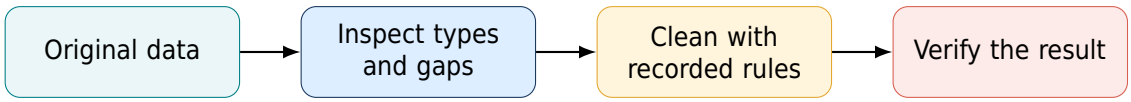



*Figure 3.4.* A repeatable workflow for preparing tabular data.

## Review Questions

1. Differentiate between a Series and a DataFrame.
2. Why is data cleaning necessary?
3. Name four useful Pandas methods.
4. Choose an appropriate chart to compare marks in several subjects and explain why.

Chapter 4

# Neural Networks and Machine Learning Tools

## 4.1 Artificial Neural Networks

An Artificial Neural Network (ANN) is a computational model inspired in a broad way by networks of connected neurons. In a typical neural network, units receive numerical inputs, apply weights and a bias, pass the result through an activation function, and send outputs to the next layer. Networks can learn patterns by adjusting parameters during training.

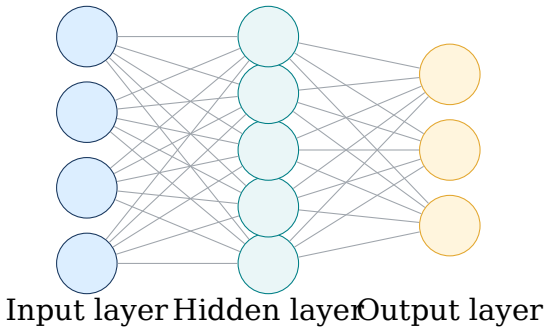



*Figure 4.1.* A simplified fully connected neural network. The drawing is conceptual; real networks can contain many layers and different connection patterns.

### 4.1.1 Training and prediction

During training, a network processes examples and compares its output with the expected target. A loss function measures the difference, and an optimisation method adjusts model parameters to reduce that loss. After training, the model can make predictions on new inputs. Performance on training examples alone is not enough; evaluation on separate data helps estimate how well the model generalises.

### 4.1.2 Weights, activation functions and loss

A weight controls how strongly an input contributes to a unit's calculation. A bias shifts the calculation, and an activation function transforms the result. Common activation functions include ReLU for many hidden layers and sigmoid for some binary-output tasks. The choice depends on the model design and the problem being solved.

A loss function measures how far a model's predictions are from the expected targets during training. The training process adjusts weights and biases to reduce the loss. A lower training loss does not automatically mean that a model will perform well on unfamiliar examples, so evaluation must use data that was not used to fit the model.

### 4.1.3 Overfitting and generalisation

Overfitting occurs when a model learns details or noise specific to its training examples and performs poorly on new data. A model that is too simple may also fail to capture useful patterns; this is often described as underfitting. Careful data splitting, suitable model complexity and evaluation on representative examples help assess generalisation. The test set should not be repeatedly used to tune every decision, because that can make the final estimate overly optimistic.

## 4.2 Applications of neural networks

Neural networks are used in image and speech recognition, handwriting recognition, language processing, recommendation systems, forecasting and selected healthcare applications. Face recognition can support device authentication; speech-to-text systems convert spoken language into text; recommendation systems rank items that may interest a user. Such applications can make errors, so human review and suitable safeguards remain important, especially in high-impact settings.

### 4.2.1 Responsible use

Data quality, privacy, fairness and transparency affect the reliability of an AI application. A model trained on unrepresentative examples may perform poorly for some groups. Sensitive personal data should be handled carefully, and a prediction should not automatically be treated as a fact. The system's purpose, limitations and evaluation results should be documented.

### 4.2.2 Privacy, fairness and human oversight

Responsible AI development begins with asking whether the data is needed and whether it may be used for the intended purpose. Personal details should not be collected simply because they are available. Access should be limited, files should be protected, and information should not be shared without appropriate permission.

Fairness requires checking whether the examples represent the people and situations in which the system will be used. An error can affect groups differently, especially when the training data is incomplete or unbalanced. A responsible project explains known limitations and provides a way for a person to review important decisions. AI outputs should be treated as information to assess, not as unquestionable truth.

## 4.3 Machine learning tools

Python's data-science ecosystem includes tools for different stages of an AI workflow. NumPy supports numerical arrays; Pandas supports tabular data handling; Matplotlib supports visualisation; Scikit-learn provides common machine-learning algorithms and evaluation utilities; TensorFlow and Keras support the development of neural-network models; XGBoost provides optimised gradient-boosting methods.

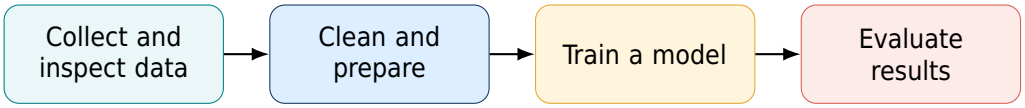



*Figure 4.2.* A simplified machine-learning workflow.

### 4.3.1 How a neural network makes a prediction

A neural network transforms input values through a series of connected calculations. In a simple unit, inputs are multiplied by weights, combined with a bias and passed through an activation function. The output of one layer becomes the input to the next layer. During training, the model adjusts its parameters using examples and a learning procedure.

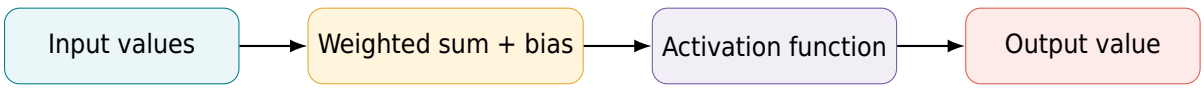



*Figure 4.3.* A simplified view of the calculation performed by a neural-network unit.

### 4.3.2 Training, validation and testing

A dataset is often divided into separate parts for training, validation and testing. Training examples are used to fit model parameters. Validation examples help compare model settings during development. Test examples are held aside for a final evaluation after important choices have been made. The exact split depends on the amount and structure of the available data, and time-based data may require a time-aware split.

If examples from the same person or event appear in both training and test sets, the test result may be overly optimistic. This is a form of data leakage. Data preparation steps such as scaling should also be fitted using training data only and then applied consistently to validation and test data. A fair evaluation depends on keeping information from the test set out of the training process.

### 4.3.3 Classification and regression

Classification predicts a category, such as whether an image contains a cat or a dog. Regression predicts a numerical value, such as an estimated temperature. Accuracy can be useful for some classification tasks, but it may be misleading when one category is much more common than another. Precision and recall can provide additional information when false positives and false negatives have different consequences.

For regression, measures such as mean absolute error describe the typical size of prediction errors in the units of the target. The chosen measure should be explained in plain language. A model should also be compared with a simple baseline, such as always predicting the average, to check whether the more complex method adds value.

### 4.3.4 Limitations of neural networks

Neural networks may require substantial data and computing resources, and their behaviour can be difficult to explain. They can reproduce patterns and biases in training data. A high score on one dataset does not guarantee the same performance in a different school, region, language or time period. Monitoring, human review and clear limits on use are important where mistakes could affect people.

### 4.3.5 Practical comparison exercise

Imagine a system that predicts whether a learner may need additional help based on practice quiz results. Discuss what the inputs might be, which information should not be collected, and how the system could be checked for errors. Explain why a teacher should review the context before acting on a prediction. The goal is to evaluate responsible design, not to label any real learner.

## Review Questions

1. Name the main layers in a simple ANN.
2. What is the purpose of a loss function?
3. Give three applications of neural networks.
4. Match NumPy, Pandas, Matplotlib and Scikit-learn with their common uses.
5. Why should a model be evaluated on data not used for training?

Chapter 5

# AI Project

## 5.1 Project guidelines

An AI project begins with a clear problem rather than with a tool. The problem statement should explain who experiences the problem, why it matters and what a useful result would look like. A manageable project has a defined scope, accessible data, a realistic schedule and criteria for evaluating the result.

### 5.1.1 A practical project cycle

A typical project cycle includes identifying a problem, defining the objective, collecting or selecting data, understanding and preparing the data, choosing an approach, building a prototype, testing it, evaluating outcomes and presenting findings. The stages may be revisited when testing reveals an issue.

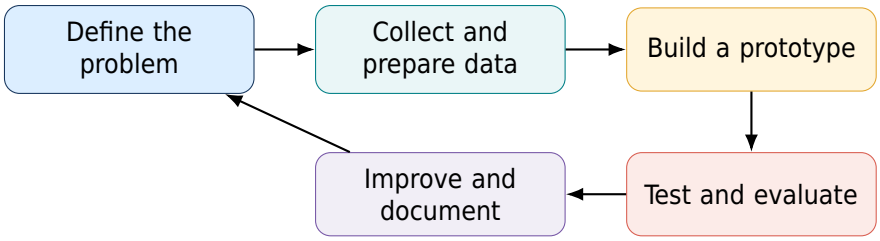



*Figure 5.1.* An iterative project cycle. Evaluation may lead to improvements or a revised problem definition.

## 5.2 Project formats and documentation

A project report should communicate the problem, objective, data source, method, implementation, evaluation, limitations and conclusion. Include diagrams or tables when they make the method easier to understand. Code should be organised, named clearly and accompanied by instructions for running it. Data sources and external materials should be acknowledged.

### 5.2.1 Choosing evaluation measures

The measure depends on the task. A classification project may report accuracy and, where appropriate, precision, recall and a confusion matrix. A numerical prediction task may use an error measure such as mean absolute error. A visualisation project should be assessed for correctness, readability and whether it answers the original question. A single score rarely tells the whole story.

## 5.3 Project review

A useful review considers whether the project solves the stated problem, whether the data and method are appropriate, whether the result can be reproduced, and whether limitations are acknowledged. Reviewers should consider privacy, possible bias, safety and the consequences of incorrect predictions. Clear documentation makes the project easier to understand and maintain.

| Review area | Questions to consider |
|---|---|
| **Problem and scope** | Is the objective clear and achievable? |
| **Data** | Are the source, quality, permissions and limitations recorded? |
| **Method** | Is the selected approach suitable for the task? |
| **Evaluation** | Are the measures relevant and the results honestly reported? |
| **Responsible use** | Have privacy, bias and potential harm been considered? |
| **Documentation** | Can another person understand and reproduce the work? |

## 5.4 Sample project: student marks analysis

A small marks-analysis project is a useful example of a data-focused AI learning task. A learner can create a table of fictional student marks, inspect the values with Pandas, calculate summaries with NumPy and present subject comparisons using Matplotlib. This project demonstrates data handling and communication; it should not be described as an AI prediction model unless a learning method is actually implemented and evaluated.

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Subject": ["English", "Maths", "Science"],
    "Marks": [76, 88, 81]
})
print("Average:", data["Marks"].mean())
plt.bar(data["Subject"], data["Marks"])
plt.title("Sample Marks by Subject")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

The values are illustrative, not real student records. A report should describe the data, explain each step, include the chart, note limitations and avoid identifying real learners without a valid reason and appropriate permission.

### 5.4.1 Planning the work and presenting results

Before writing code, prepare a short plan with the project objective, required inputs, expected outputs and a list of tasks. Divide the work into manageable steps, such as preparing a data file, checking its quality, performing calculations, creating a chart and writing a conclusion. A small prototype should be tested early so that major problems are discovered before the final report is prepared.

The final presentation should explain the original question, the approach used and the evidence that supports the conclusion. Screenshots or diagrams can help readers follow the process, but they should be relevant and clearly labelled. If the result is uncertain or the dataset is small, state that limitation rather than making a broader claim than the evidence supports. Keep a copy of the code and note the software libraries required to run it.

### 5.4.2 Defining a measurable project objective

A good objective describes the problem, the intended user and the result that would count as useful. “Use AI to improve education” is too broad to guide a small project. “Create a chart that compares fictional marks across three subjects and explain the

highest and lowest values" is specific and can be completed with a small dataset. If the project claims to predict an outcome, the report must explain how the prediction is trained and evaluated.

Before selecting a tool, ask whether AI is necessary. A clear set of rules or a simple chart may solve the problem more transparently than a machine-learning model. The best project is not necessarily the one with the most complex technology; it is the one that addresses the stated need safely and convincingly.

### 5.4.3 Data permissions and documentation

Use data that is permitted for the project. For classroom practice, fictional or appropriately anonymised examples are often sufficient. Do not upload personal records to online services unless there is a valid reason and appropriate permission. Keep track of the data source, collection date, fields, units, cleaning decisions and known limitations. If external code or images are used, acknowledge the source and respect its licence.

### 5.4.4 A practical project schedule

Divide a project into small tasks: define the question, prepare data, implement the method, test the result, create a visual explanation and write the report. Set a completion check for each task. Keep the scope realistic so that enough time remains for testing and documenting the work. If a step takes longer than expected, simplify optional features rather than skipping evaluation.

### 5.4.5 Interpreting results honestly

A result should be described in proportion to the evidence. A small sample may illustrate a method but cannot represent every learner or community. A correlation between two measurements does not prove that one causes the other. If the output changes substantially when one data point is removed, mention that sensitivity. Include failures and limitations as part of the report, not as details to hide.

### 5.4.6 Project report structure

A useful report can contain a title, problem statement, objective, data description, method, tools, implementation steps, results, evaluation, limitations, responsible-use considerations and conclusion. Each section should help the reader understand a different part of the work. Diagrams should be labelled, code should be formatted clearly, and every figure should be discussed in the text. A conclusion should answer the original question and avoid claims that were not tested.

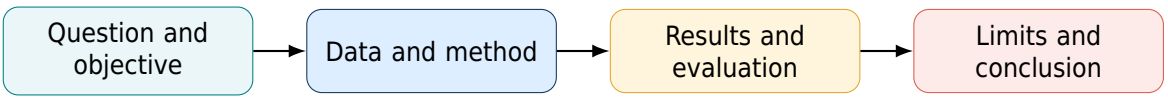



*Figure 5.2.* A logical structure for explaining an AI project.

### 5.4.7 Reflection after completion

After completing the project, review whether the original objective was met, what evidence supports the result and which steps were difficult. Identify one improvement that would make the result more reliable or easier to use. A useful reflection is specific: it describes what happened and what would be changed next time, rather than simply saying that the project was good or bad.

## Review Questions

1. List the major stages of an AI project.
2. Why is the problem statement important?
3. What should a project report contain?
4. Explain why evaluation measures depend on the task.
5. Identify two privacy or fairness questions to consider before using data.